VinUniversity

Advanced Agent Architectures

AICB-P2T3 · Ngày 16 · Chương 4 — Agent Nâng Cao

Giảng viên

VinUniversity · Phase 2 · Track 3 · Tuần 4

Hook

Tại sao Reflexion agent giải quyết được bài toán mà ReAct không làm được?

Hôm nay ta sẽ trả lời câu hỏi này bằng benchmark, pattern và demo code.

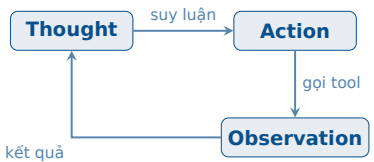
Agenda

1. Khi nào single-agent thất bại?
2. Reflexion: thêm self-evaluation vào loop
3. LATS, Voyager và decision matrix
4. Kỹ thuật nâng cao trước khi vào lab
5. Demo + lab + auto-grading

Khi nào Single Agent thất bại?

ReAct — mạnh nhưng không biết sửa lỗi

ReAct — Reasoning + Acting

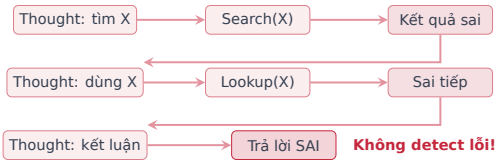


- Xen kẽ **Reasoning** (suy nghĩ) + **Acting** (hành động)
- Agent tự quyết định: gọi tool nào, khi nào dừng
- Đã học ở GĐ1 — nền tảng cho mọi agent pattern

Nhắc lại

ReAct = “Think before you act” — mỗi bước agent giải thích lý do trước khi hành động

ReAct thất bại khi nào?



3 failure modes chính:

1. **Lỗi lan tỏa**: Sai ở bước 1 → sai hết chuỗi
2. **Infinite loop**: Tool trả noise → agent lặp mãi
3. **Không backtrack**: Đi sai đường nhưng không quay lại

Lưu ý

Root cause: ReAct **không có cơ chế tự đánh giá**. Khi đi sai, không có signal nào báo "dừng lại, suy nghĩ lại".

Bài học production 2025–2026: đừng nâng cấp agent quá sớm

- Nhiều bài toán thực tế chỉ cần **retrieval + tools + structured output**
- Bắt đầu bằng **single-agent** hoặc workflow đơn giản, sau đó mới thêm complexity
- Chỉ chuyển sang multi-agent khi có **tool overload**, prompt quá nhiều nhánh logic, hoặc cần **specialist ownership**

Thông điệp cho học viên

Đừng học pattern theo kiểu “càng nhiều agent càng tốt”. Hãy chọn pattern theo **mức độ cần thiết** của task và chi phí vận hành.

Ví dụ thực tiễn: PR review agent bị lỗi lan tỏa thế nào?

1. Agent đọc sai module trong diff ngay từ bước đầu
2. Gọi checker / search trên file không liên quan
3. Tổng hợp evidence sai nhưng vẫn tự tin kết luận
4. ReAct thường không có signal rõ để tự dừng và sửa

Vì sao Reflexion giúp hơn?

Evaluator có thể chấm: “kết luận chưa grounded vào diff và test logs”. Reflector biến lỗi đó thành chiến lược mới cho lần chạy tiếp theo.

Bằng chứng: ReAct struggle với multi-hop reasoning

35.1%
ReAct EM
trên HotpotQA

Cao
Fail rate
trên multi-hop

0
Số lần
agent tự sửa lỗi

Câu hỏi then chốt

Nếu thêm cho agent khả năng **tự đánh giá** kết quả và **rút bài học** từ sai lầm thì sao? → Đó chính là ý tưởng của **Reflexion**.

Reflexion — Dạy Agent tự phản tỉnh

Thêm self-evaluation vào reasoning loop

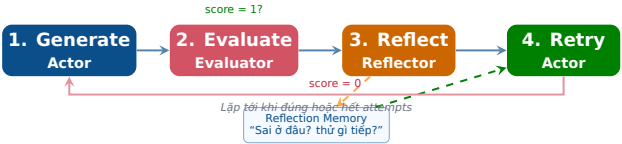
Ý tưởng cốt lõi

Reflexion (Shinn et al. 2023)

Thêm 2 thành phần vào ReAct: **Evaluator** (đánh giá kết quả) và **Reflector** (rút bài học). Agent thử, đánh giá, suy ngẫm, rồi thử lại — giống cách con người học từ sai lầm.

Analogy: Như sinh viên làm bài thi. Lần 1 sai → xem đáp án, hiểu tại sao sai → lần 2 làm đúng. ReAct chỉ làm 1 lần rồi nộp. Reflexion cho phép "xem lại bài" và sửa.

Kiến trúc Reflexion — 4 bước



3 vai trò LLM

Actor: sinh hành động
Evaluator: chấm đúng/sai
Reflector: rút bài học

Điểm khác biệt vs ReAct

Dùng **text feedback** thay vì gradient.
Critique bằng ngôn ngữ tự nhiên nên dễ parse, debug và benchmark.

Reflexion State — Code-Level

Python schema

```
class ReflexionState(TypedDict):
  messages: list[BaseMessage]
  trajectory: list[str]
  reflection_memory: list[str]
  attempt_count: int
  success: bool

REFLECT = ``failed because: {error}''
      ``lesson: {lesson}''
      ``next strategy: {strategy}''
```

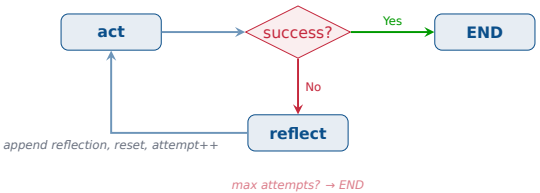
5 thành phần state:

1. `messages`: hội thoại hiện tại
2. `trajectory`: lịch sử hành động
3. `reflection_memory`: bài học rút ra
4. `attempt_count`: số lần thử
5. `success`: đã đúng chưa?

Lưu ý

Dùng **sliding window**: quá ngắn thì quên, quá dài thì tốn context.

Reflexion trong LangGraph



Node “reflect”

1. Lấy trajectory (đã làm gì?)
2. Gọi Reflector LLM (sai ở đâu?)
3. Append reflection vào memory
4. Reset messages, tăng attempt

Termination

Dừng khi: success = True
Hoặc: attempt $\geq$ max (default 3)

Tránh infinite loop — cái mà ReAct gặp phải

Evaluator prompt nên được thiết kế thế nào?

```python
class JudgeResult(BaseModel):
    score: int
    reason: str
    missing_evidence: list[str]
    spurious_claims: list[str]
```

- Output nên là **structured** thay vì free-form
- Score phải đi kèm **reason** và **evidence gap**
- Nếu evaluator quá vague, reflection sẽ không actionable

Best practice

Dùng Pydantic/JSON schema để lab dễ parse, benchmark và auto-grade.

Reflection memory: ghi gì, bỏ gì?

- **Nên ghi**: failure reason, lesson, next strategy, evidence titles
- **Không nên ghi**: toàn bộ trace dài dòng nếu không giúp lần thử sau
- Có thể dùng **sliding window** hoặc **memory compression**

Teaching point

Memory tốt là memory **ngắn, cụ thể, hành động được**. Không phải memory càng dài càng tốt.

Reflexion failure modes trong production

1. **Evaluator bias**: tự chấm quá dễ hoặc quá khắt khe
2. **Reflection drift**: bài học chung chung, không giúp được attempt sau
3. **Context bloat**: reflection memory chiếm hết context window
4. **Cost blow-up**: accuracy tăng ít nhưng chi phí tăng mạnh

Thông điệp

Reflexion không miễn phí. Cần đánh giá **accuracy gain so với cost/latency increase**.

Reflexion cải thiện đáng kể

91%
HumanEval
(code gen)

80%
HotpotQA
(multi-hop QA)

+20–30%
Cải thiện
vs ReAct

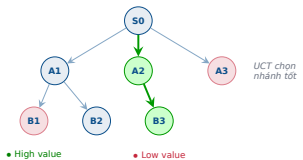
Tại sao hiệu quả?

Reflexion dùng **episodic memory** — agent "nhớ" bài học từ các lần thử trước *trong cùng episode*. Giống cách bạn nhớ "lần trước đã thử cách này không được, lần này thử khác".

Bức tranh rộng hơn

LATS, Voyager và khi nào nên dùng agent phức tạp

LATS — Khi cần tìm đường tối ưu



LATS

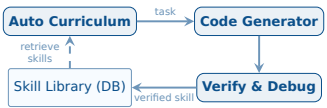
MCTS + LLM: mỗi node là một trạng thái suy luận; LLM đóng vai *policy*, *value* và *simulation*.

- Chính xác hơn Reflexion (92.7% vs 91%)
- **Nhưng** tốn gấp 3–5× compute
- Cần environment cho phép **undo**

Lưu ý

Chỉ đáng dùng khi task có giá trị cao và có thể rollback, như code gen hoặc game.

Voyager — Agent tích lũy kỹ năng



- Agent tự đặt mục tiêu, viết code, lưu skill đã verified
- Skill mới xây trên skill cũ (**compound learning**)
- Sau 3h: 63 skills vs 7 của AutoGPT

Ứng dụng thực tế

Hợp với code generation, DevOps automation và các domain cần tích lũy "thư viện kinh nghiệm" qua nhiều episode.

Khi nào dùng pattern nào?

Pattern	Memory	Chi phí	Accuracy	Khi nào dùng?
ReAct	Không	$	Baseline	Task đơn giản, 1 bước
Reflexion	Episodic	$$	+20–30%	Multi-step, cần self-correct
LATS	Tree	$$$$$	+∼2%	High-stakes, cho phép undo
Voyager	Persistent	$$$	N/A	Open-ended, cần tích lũy

Lưu ý

Nhiều bài toán thực tế **không cần agent**: retrieval, template fill, structured output đã đủ. Đọc: *"AI Agents That Matter"* (2024) — đừng over-engineer.

Case study mới: multi-agent research system

- Một planner agent chia câu hỏi thành nhiều sub-questions
- Các worker agents tìm thông tin song song
- Một synthesizer agent hợp nhất và viết câu trả lời cuối cùng
- Pattern này hợp với **open-ended research**, không phải mọi business workflow

Lesson

Multi-agent có ý nghĩa khi bài toán **mở**, khó dự đoán trước các bước, và có lợi từ parallel exploration.

Checklist triển khai an toàn cho agent nâng cao

1. Có `max_attempts`
2. Có structured outputs cho evaluator / tools
3. Có trace để debug từ sớm
4. Tool càng deterministic càng tốt
5. Có human review cho action rủi ro

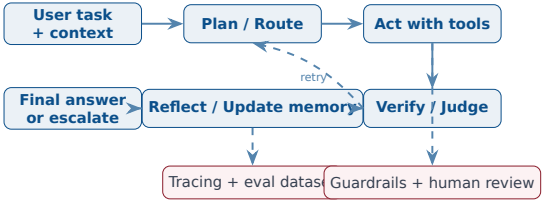
Production mindset

Prompt chỉ là 1 phần. Còn lại là state, tool quality, tracing, eval và guardrails.

Kỹ thuật nâng cao trước khi vào lab

Các pattern production giúp agent ổn định, dễ debug và dễ đánh giá hơn

Template kiến trúc agent production-ready



- Đừng dừng ở prompt + tool loop
- Thêm **judge**, **memory**, **trace** và **guardrails**
- Tách rõ phần *reasoning* với phần *execution*
- Chuẩn hóa state để benchmark và auto-grade dễ hơn

Rule of thumb

Evaluator tốt quyết định chất lượng Reflexion

Evaluator nên chấm 4 thứ

1. **Correctness**: câu trả lời có đúng không?
2. **Grounding**: có bám evidence/tool output không?
3. **Completeness**: đã trả lời đủ các phần chưa?
4. **Actionability**: reflection có thể sửa được không?

Anti-pattern

Nếu evaluator chỉ nói ``incorrect'' thì reflector rất khó sinh bài học hữu ích.

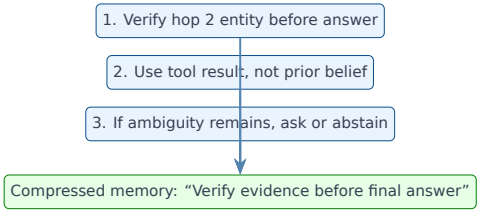
Ví dụ output có cấu trúc

```
{
"is_correct": false,
"failure_mode": "missed-hop",
"evidence_used": ["wiki:person_a"],
"feedback": "Bạn đã đúng hop 1 nhưng
chưa verify hop 2.",
"next_action": "search_second_hop"
}
```

Thiết kế tốt

Structured output giúp log, filter theo failure mode và chuyển thẳng sang auto-grading/reporting.

Reflection memory: lưu *bài học*, không lưu nguyên chat history



Memory entry nên ngắn và thao tác được

`lesson`: “Luôn verify thực thể ở hop 2 trước khi trả lời”
`trigger`: “Câu hỏi 2-hop có entity dễ nhầm”
`fix`: “Search thêm 1 bước và so khớp tên riêng”

- Mỗi entry chỉ 1 lỗi + 1 cách sửa

Điểm dạy học quan trọng

Memory tốt làm **giảm lặp lỗi**, nhưng memory dài quá lại làm prompt noisy và tăng cost.

Plan - Act - Verify tách bạch sẽ ổn định hơn loop “nghĩ rồi làm luôn”



- **Plan**: liệt kê 2–4 subgoals quan sát được
- **Act**: chỉ gọi 1 tool phù hợp cho mỗi subgoal
- **Verify**: kiểm tra output có thật sự giải quyết subgoal không
- Tránh để model “nhảy cóc” từ plan sang final answer

Độ tin cậy của tool quan trọng không kém độ mạnh của model

5 kỹ thuật tăng reliability

1. Schema chặt cho input/output tool
2. Retry có điều kiện cho lỗi tạm thời
3. Idempotent action cho thao tác ghi dữ liệu
4. Timeout + fallback khi tool treo
5. Risk tiering: read-only vs write/high-impact

Production rule

Tool nào ghi dữ liệu, gửi email, thanh toán, xóa bản ghi... nên có human checkpoint hoặc approval gate.

Prompt/tool contract

Good: "Nếu search không trả evidence rõ ràng, không được đoán; trả về `insufficient`.

Bad: "Cố gắng trả lời bằng mọi giá."

Nhìn dưới góc dạy học

Sinh viên thường tối ưu prompt trước, nhưng bug thực tế hay nằm ở tool schema, parser, timeout, retry và side effects.

Observability + eval flywheel: cách agent tiến bộ sau mỗi lần demo



- Đừng chỉ log final answer
- Hãy log cả decision, tool calls, retry và failure modes
- Từ trace mới sinh ra dataset chấm tự động có ích

Liên hệ với lab

`report.json`, `runs.jsonl` và breakdown failure modes chính là phiên bản mini của eval flywheel này.

Khi chưa cần multi-agent

Thử 4 bước này trước

1. Router hoặc tool policy tốt hơn
2. Plan-then-act + verifier
3. Reflexion + memory + eval
4. Guardrails + approval gates cho tool có side effects

Vì sao?

Nhiều lỗi tưởng là do “thiếu agent chuyên gia” nhưng thực ra đến từ tool schema kém, thiếu evaluator hoặc thiếu trace.

- Nếu chỉ có 1 domain tool chính, hãy giữ single-agent
- Nếu task là workflow cố định, prompt chaining hoặc routing thường đủ
- Nếu lỗi chính là hallucination, hãy sửa grounding/evaluator trước

Heuristic

Complexity chỉ nên tăng sau khi bạn đã có benchmark baseline và biết rõ bottleneck nằm ở đâu.

Khi multi-agent bắt đầu đáng tiền

Dấu hiệu phù hợp

- Nhiều domain tool rất khác nhau
- Cần **parallel exploration** cho open-ended research
- Cần tách vai trò **planner / worker / judge / synthesizer**
- Cần tách **read/write agents** để giảm risk

Ví dụ phù hợp

Research system, code review + synthesis, ops assistant có approval workflow.

Quy tắc an toàn

Càng nhiều agent, càng cần handoff contract rõ ràng: input schema, output schema, stop condition, ownership của state.

Thông điệp cuối phần lý thuyết

Phức tạp hơn không mặc định tốt hơn. Chỉ lên multi-agent khi **single-agent + tools + eval + memory** đã chạm trần rõ ràng.

Demo & Thực hành

Xem Reflexion hoạt động thực tế

Demo: ReAct vs Reflexion — Side-by-side trên HotpotQA

- Cùng câu hỏi 2-hop, chạy cả hai agent với LangSmith tracing
- ReAct: sai entity ở hop 1, lỗi lan tỏa, trả lời sai
- Reflexion: attempt 1 sai → Evaluator cho score=0 → Reflector sinh bài học → attempt 2 đúng
- So sánh: trace, accuracy, cost/query

Lab 16: Implement Reflexion agent từ scratch với LangGraph

Mục tiêu: Reflexion agent repo + benchmark report (EM comparison, cost analysis, failure categorization)

Thời lượng: 2 giờ

Lab 16 — Các bước thực hành

1. **Build state machine**: nodes = [act, evaluate, reflect, terminate], edges có conditional routing
2. **Build Evaluator**: LLM-as-Judge, prompt yêu cầu score 0–1 + reason, parse output Pydantic
3. **Add reflection memory**: mỗi entry = (attempt_id, failure_reason, lesson, strategy_next)
4. **Benchmark**: Chạy Reflexion vs ReAct trên 20 câu HotpotQA — đo EM, attempts, token cost

Deliverable

GitHub repo + benchmark report: bảng so sánh EM, phân tích cost, phân loại failure modes

Lab roadmap 120 phút

1. **30 phút**: chạy ReAct baseline và hiểu trace
2. **35 phút**: thêm Evaluator dạng structured output
3. **25 phút**: thêm Reflector + reflection memory
4. **30 phút**: benchmark, viết report, sinh artifact để auto-grade

Instructor tip

Cho học viên chạy mock mode trước để hiểu format output, sau đó mới thay provider thật.

Bonus tasks để phân hoá học viên

- adaptive max attempts
- memory compression
- evidence-grounded evaluator
- mini-LATS branching (2 candidates / step)
- plan-then-execute trước khi reflect

Cách chấm

Không chỉ chấm “có làm được không” mà chấm thêm **thí nghiệm, trade-off và giải thích**.

Deliverable schema để dễ chấm tự động

- `report.json`: metric tổng hợp
- `report.md`: narrative analysis
- `react_runs.jsonl`, `reflexion_runs.jsonl`: trace theo từng câu hỏi
- Giữ schema ổn định để TA chấm objective nhanh

Outcome

Sinh viên vừa học pattern agent, vừa học tư duy **evaluation-driven engineering**.

Tổng kết

Takeaway 1

Reflexion là nâng cấp hợp lý khi ReAct thất bại: cost vừa phải, accuracy tăng rõ.

Takeaway 2

LATS và Voyager đổi compute lấy optimality hoặc generality; chỉ dùng khi task thật sự cần.

Takeaway 3

Cẩn thận "degeneration-of-thought": reflection kéo dài có thể làm output tệ hơn.

Takeaway 4

Xu hướng production: structured outputs, tracing và eval quan trọng hơn free-form reasoning.

Ngày 17: Memory Systems for Agents

Agent đã biết reasoning — nhưng tại sao nó quên hết sau mỗi conversation?

- Hoàn thành Lab 16: Reflexion agent + benchmark
- Đọc: Anthropic “Building Effective Agents”

Q&A

Reflexion có phải luôn tốt hơn ReAct? Khi nào nên dùng cách nào?

Cảm ơn!

AICB-P2T3 · Ngày 16 · Advanced Agent Architectures

`github.com/vinuni-aicb`

Liên hệ: `instructor@vinuni.edu.vn`